

Gymnázium Praha 6, Arabská 14

Programování



Šimon Petříček, Reuel Dejmal, Jakub Hruška

2026

Ročníkový projekt 3.E

Předmět: Programování

Téma: Cell Expansion Wars

Autor: Šimon Petříček, Reuel Dejmal, Jakub Hruška

Třída: 3.E

Školní rok: 2025/26

Vedoucí práce: Mgr. Jan Lána

Třídní učitel: Mgr. Jan Tuzar

Poděkování

Rád bych poděkoval Mgr. Jan Lána za jeho čas, cenné rady a pomoc při úpravách naší práce.
Také děkuji celému týmu za dobře odvedenou práci.

Prohlášení

Prohlašuji, že jsem jediným autorem tohoto projektu, všechny citace jsou řádně označené a všechna použitá literatura a další zdroje jsou v práci uvedené. Tímto dle zákona 121/2000 Sb. (tzv. Autorský zákon) ve znění pozdějších předpisů uděluji bezúplatně škole Gymnázium, Praha 6, Arabská 14 oprávnění k výkonu práva na rozmnožování díla (§ 13) a práva na sdělování díla veřejnosti (§ 18) na dobu časově neomezenou a bez omezení územního rozsahu.

V Praze dne 14.5. 2025

Podpis autorů

Obsah:

Úvod.....	5
1. Popis Hry.....	5
1.1 Herní myšlenka.....	5
1.2 Typy buněk.....	6
1.3 Herní mechaniky.....	6
1.4 Podmínky výhry a prohry.....	7
1.4 Úrovně a obtížnosti.....	8
2. Technická část.....	9
2.1 Použité technologie.....	9
3.2 Struktura souborů.....	10
3.3 Frontend - popis herní logiky.....	11
3.4 Backend - popis serverové části.....	13
3.5 Komunikace frontendu a backendu.....	14
4. Grafika a design.....	15
5. Využití AI při vývoji.....	18
6. Podobné hry srovnání.....	20
6.1 Podobné hry na trhu.....	20
7. Závěr.....	20
7.1 Co jsme se naučili.....	20
7.2 Co by šlo vylepšit.....	21
7.3 Zhodnocení projektu.....	21
Zdroje.....	22

Anotace

Tento projekt představuje webovou hru s názvem Cell Expansion Wars. Hra nabízí tři herní úrovně a multiplayer režim pro více hráčů. Součástí aplikace je také systém přihlašování, který zajišťuje bezpečnost a ukládání herních výsledků.

Promyšlená herní mechanika zajišťuje dlouhodobé udržení pozornosti hráče. Celkový koncept vychází z osvědčených herních systémů a moderního přístupu k návrhu her.

Anotace

This project presents a web-based game called Cell Expansion Wars. The game offers three levels for players to explore, as well as a multiplayer mode supporting multiple players simultaneously. The application also includes a login system to ensure security and track player results.

Thoughtfully designed game mechanics keep players engaged for extended periods of time. The overall concept draws from proven game design systems combined with a modern approach to game development.

Úvod

Hraní her patří mezi nejčastější způsoby trávení volného času, a to jak v online prostoru, tak i v reálném světě. V posledních letech byl zaznamenán výrazný nárůst her sloužících jako odreagování po práci. Pod pojmem odreagovací hra si můžeme představit hru, při které hráč dokáže vypnout svoji mysl a načerpat novou energii. Na druhé straně, a trochu protikladně, klademe důraz na strategické myšlení a aktivní zapojení mozku. Kombinování těchto dvou podmínek splňuje jen málokterá hra, a to právě kvůli jejich zdánlivě protichůdným přístupům.

Naše hra dokázala tyto dvě podmínky propojit. Pracujeme na ní jako na našem ročníkovém projektu a v průběhu jeho prezentace si budeme představovat metody a části kódu použité při jejím vývoji. Popis hry je velmi jednoduchý: hráč ovládá 1 až 3 buňky a postupně z nich vysílá své vojáky, aby dobývali okolní buňky. S pomocí grafiky, kterou hra nabízí, si hráč může odpočinout, ale pokud chce zvítězit, musí zapojit strategické myšlení.

Pokud se podíváme na projekt z technického hlediska je projekt rozdělen na frontend a backend. Javascript zajišťuje herní logiku. Vojáci se pohybují plynule po obrazovce a po dosažení cílové buňky posílí vlastní buňku, nebo ničí cizí ostatní buňky. Při zničení nepřátelské buňky, tak se buňka přebarví. Hra obsahuje tři typy buněk: bílé (hráč), černé (počítač) a modré (neutrální). Každý herní level má odlišným rozmístění a počet buněk. Backend je postaven na Django s asynchronními pohledy používající async/await, které obstarávají registraci, přihlášení, správu profilů a vytváření multiplayerových her.

Hra nabízí tři herní levely se stupňující se obtížností a také multiplayerovým režimem pro hraní s více hráči v reálném čase. Tento projekt byl pro nás náročnou výzvou, která posunula naše celkové znalosti programování na novou úroveň.

1. Popis Hry

1.1 Herní myšlenka

Brain Cell Wars je strategická prohlížečová hra. Hráč ovládá přidáný počet mozkových buněk a snaží se dobýt ostatní buňky a získat celou herní plochu. Dobyť ostatních buněk dosáhne, vysláním svých vlastních vojáků. Životy buňky koreluje s počtem vojáků.

Ve hře si člověk může vyzkoušet taktické rozhodování a zkoušení nových strategií. Jednoduchost ovládání je klíčová pro dlouhodobé hraní u uživatelů. Při hraní musí hráč neustále zvažovat možnosti útoku a obrany.

1.2 Typy buněk

Hra nabízí tři typy buněk, které jsou od sebe odděleny barvou. Hráč může ovládat pouze jednu barvu buňky, a to jest bílou. Bílá buňka je jediná, ze které může hráč posílat své vojáky. Zbylé dvě barvy černá a modrá nemohou být ovladatelná hráčem.

Modrá buňka ukazuje neutralitu. Nepatří, ani jednomu z hráčů. Začátkem hry dostává určitý počet životů, které jsou možné pro její dobytí. Modrá buňka nabízí možnost strategického dobytí v dalším pokračování hry.

Černá buňka je nepřítel. Nepřítel pošle jednoho vojáka přibližně každou sekundu. Posílání vojáků je řízeno pomocí AI technologie a probíhá náhodně. Černá buňka může útočit, jak na bílou buňku, tak i na modrou. Barva buňky nehraje roli.

1.3 Herní mechaniky

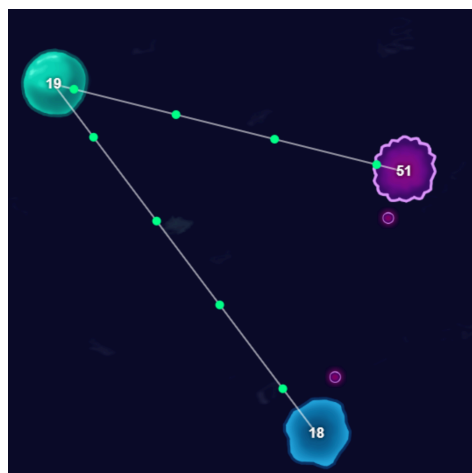
Jednoduchost hry je jejím klíčovým parametrem. Hráči stačí kliknout na svoji buňku, čímž ji vybere, a poté kliknout na cílovou buňku, kam chce zamířit útok. Mezi těmito dvěma buňkami vznikne trvalá linka, po které hráč vysílá své vojáky. Vojáci jsou odesílání automaticky v pravidelných intervalech. Kliknutím na linku hráč tuto linku zruší. Díky tomuto mechanismu nemusí hráč vysílat vojáky manuálně a stačí mu pouze nastavit správné linky a sledovat dění na hrací ploše.

V momentě, kdy hráčovi vojáci dorazí k nepřátelské nebo neutrální buňce, začínají snižovat počet jejích vojáků. Pokud počet vojáků klesne na nulu, buňka je dobytá a změní svoji barvu na bílou.

Počítač funguje na podobném principu. Nevytváří však trvalé linky, ale vysílá jednotlivé vojáky náhodným směrem přímo z každé své buňky. V případě dobytí buňky počítačem se její barva změní na černou. Jak bílá, tak černá buňka obdrží při dobytí 5 vojáků jako výchozí stav.



obrázek 1 - nepřátelský voják



obrázek 2 - trvalá linka a vojáci hráče

1.4 Podmínky výhry a prohry

Jednoduchost prostupuje celou hrou a platí i pro podmínky výhry a prohry. Každá herní úroveň sdílí stejnou podmínku vítězství. Hráč musí dobýt všechny cizí buňky, tedy jak modré neutrální, tak černé nepřátelské. Po úspěšném dokončení úrovně se barva příslušného tlačítka změní ze zelené na zlatou a v sekci úspěchů se zobrazí zlatý řádek dané úrovně. Po dokončení všech tří úrovní se hráči zobrazí výzva k vyzkoušení multiplayerového (hrajou dva a více hráčů) režimu. Multiplayer je přístupný od samého začátku hry, ovšem hráč, který má za sebou všechny tři úrovně, přistupuje do něj s výrazně lepší znalostí herních mechanik a větší šancí na úspěch.

Podmínka prohry je stejně jednoznačná. Pokud nepřátelské černé buňky dobudou všechny hráčovy bílé buňky, hra končí prohrou. Hráči se zobrazí oznámení s možností návratu do hlavního menu, odkud může hru zkusit znovu. Hra nezaznamenává počet odehraných pokusů, ale jediným cílem, který sleduje, je hráčovo vítězství. Každý nový pokus začíná za stejných podmínek a dává hráči plnou příležitost zlepšit svoji strategii a uspět.

1.4 Úrovně a obtížnosti

Hra nabízí tři singleplayerové úrovně, přičemž každou z nich může v daný moment hrát pouze jeden hráč. Jednotlivé úrovně se od sebe liší rozmístěním buněk na hrací ploše, počtem nepřátelských buněk i jejich výchozím počtem vojáků. Každá úroveň přináší nové herní situace, odlišné taktické výzvy a vyšší míru obtížnosti než úroveň předchozí. Obtížnost postupně roste skrze několik herních parametrů a zvyšuje se počet nepřátelských buněk, jejich výchozí počet vojáků, nebo naopak klesá počáteční síla hráčovy buňky. Obtížnost není možné nastavit manuálně v žádné části hry. Hráč se s rostoucí náročností musí vypořádat výhradně prostřednictvím zdokonalování vlastních taktik a herních strategií, což přirozeně prodlužuje herní zážitek a motivuje k opakovanému hraní.

Po úspěšném dokončení úrovně dojde k několika vizuálním změnám. Tlačítko dané úrovně v hlavním menu změní svoji barvu ze zelené na zlatou a zobrazí se na něm ikona poháru jako symbol splnění. Zároveň hráče odmění animace zlatých konfet padajících po herní ploše. Splnění úrovně rovněž odemkne přístup k úrovni následující, která na hráče čeká s novými výzvami a složitějšími herními situacemi. Postup hráče je průběžně ukládán, takže dosažené výsledky zůstávají zachovány i po zavření hry.

2. Technická část

2.1 Použité technologie

Projekt má dvě části frontend a backend. Každá tato část slouží k jinému účelu. Oba dva dohromady se navzájem propojují a pracují jako celek pro náš projekt.

Frontend je vytvořen třemi základními technologiemi HTML, CSS a JavaScript. HTML je kostra celé stránky a určuje všechny elementy a věci, které hráč vidí. HTML určuje, kde se nachází tlačítka pro přihlášení, herní plochu a tlačítko pro zobrazení pravidel během hry, nebo overlay obrazovky vyskakující po výhře. HTML ovšem neurčuje, vzhled věcí, tohle má na starost CSS.

CSS je druhou technologií frontendu. CSS se stará převážně o vizuální stránku hry, která ukazuje, jak různé elementy vypadají. CSS má na starosti barvy tlačítek, jejich velikost, zaoblení rohů a rozmístění prvků, který se nachází na stránce. V podstatě se stará CSS o animaci projektu.

Poslední technologií je programovací jazyk Javascript, který zajišťuje chod věcí. Pro představu stará se o pohyby vojáků po herní ploše a změnu barvy buňky. Javascript je základní kámen pro funkce a plynulost hry.

Backend je postaven na frameworku Django. Framework je sada nástrojů a pravidel, která usnadňuje vývoj webových aplikací. Django je napsáno v programovacím jazyku Python, který je znám svoji jednoduškou a bezpečností.

Pro multiplayerový režim jsou použity Django Channels a server Daphne. Tyto nástroje umožňují okamžitou komunikaci mezi hráči, která probíhá bez většího zpoždění. Pro ukládání stavu hry mezi hráči je použit Redis jako rychlá databáze s možností uchování dat v paměti počítače. Na rozdíl od klasické databáze která ukládá data na pevný disk, Redis uchovává data přímo v operační paměti serveru, což zajišťuje výrazně rychlejší přístup. Django Channels rozšiřuje základní Django o podporu WebSocket protokolu, který umožňuje obousměrnou komunikaci mezi serverem a prohlížečem hráče. Kombinace těchto tří technologií tvoří základ celé multiplayerové infrastruktury projektu.

3.2 Struktura souborů

Projekt se zakládá na několika hlavních souborech, kde každý soubor má svoji unikátní roli. Toto rozdělení usnadní práci a opravu případných chyb.

Frontend se skládá z 4 hlavních souborů. Prvním z nich je `index.html`, která zobrazuje první stránku, kterou hráč uvidí po otevření hry. Obsahuje hlavní menu se 3 úrovní, přihlášením a registrací. Rovněž obsahuje v levém horním rohu tlačítko pro zobrazení úspěchů a tlačítko pro multiplayera.

V `game.html` se nachází, na které se odehrává samotná hra. Soubor obsahuje herní plochu, ve kterém se vykreslí buňky a vojáci. Soubor rovněž obsahuje obrazovky pro zobrazení výhry nebo prohry. Pro lepší herní zážitek obsahuje hra tlačítko pro pravidla.

Soubor `main.js` je z celého frontendu největší a nejdůležitější. Obsahuje veškerou herní logiku od pohybu vojáků po herní ploše, dobývání buňky, chování nepřátelské ai buňky po rozpoznání výhry a prohry. Main soubor se rovněž stará o definici všech úrovní včetně rozmístění buněk a jejich výchozího počtu vojáků. Tento soubor je doopravdy hlavní oporou celého projektu.

Poslední hlavním souborem frontendu je `style.css`, který definuje jak vypadá celá hra. Soubor obsahuje barvy tlačítek, velikost písma, animace a rozmístění všech prvků na stránce. Tento soubor stojí za tím, proč mají tlačítka úrovní 3 různé vizuální stavy. Soubor vyobrazuje tři barvy úrovní. Šedou pro zamčené úrovně, zelená pro odemčené a zlaté pro dokončené. Soubor obsahuje plynulé scrollování mezi sekcemi hlavní stránky.

Backend se rovněž skládá ze 4 hlavních souborů. Prvním souborem je `views.py`, který obsahuje funkce, které se spustí pokud hráč vytvoří nějakou akci např. přihlášení nebo vytvoření nové multiplayerové hry. Tyto funkce zpracují požadavek hráče, provedou potřebné operace s databází a pošlou mu odpověď. Všechny funkce jsou napsány jako paralelní pomocí klíčových slov `async` a `await`, což znamená že server může obsluhovat více hráčů zároveň bez čekání a s touto pomocí zvládnou aplikace větší počet hráčů bez zpomalení.

Soubor `models.py` definuje jak jsou data uložena v databázi. Databáze je místo kde jsou trvale uloženy informace o hráčích a jejich hrách. Každý model v tomto souboru odpovídá jedné tabulce v databázi, jako například tabulka hráčů, tabulka odehraných her nebo tabulka herních akcí. Tento soubor je základ celé datové vrstvy aplikace a bez něj by nebylo možné ukládat žádné informace o hráčích ani jejich postupu.

Soubor `urls.py` funguje jako telefonní seznam aplikace. Obsahuje seznam URL adres a ke každé adrese přiřazuje funkci která se má spustit když hráč tuto adresu navštíví. Například adresa `/login/` spustí funkci pro přihlášení a adresa `/register/` spustí funkci pro registraci. Bez tohoto souboru by server nevěděl co má dělat když hráč navštíví určitou adresu a vrátil by chybu.

Soubor `settings.py` obsahuje základní nastavení celé aplikace, jako například jak se připojit k databázi, jaké bezpečnostní klíče se používají nebo jaká rozšíření jsou nainstalována. Také definuje že aplikace používá server Daphne pro zpracování WebSocket spojení místo standardního Django serveru. Bez správného nastavení v tomto souboru by celá aplikace odmítla spustit nebo by nefungovala správně. Tento soubor se nastavuje pouze jednou na začátku projektu a během vývoje se mění jen zřídka.

3.3 Frontend - popis herní logiky

Frontend je část aplikace, která běží přímo v prohlížeči hráče, to co hráč vidí. Veškerá herní logika hry je napsána v `main.js`, jak už bylo zmíněno. Hra se vykresluje s pomocí HTML5 Canvas. Javascript pomocí tohoto plátna vykreslí každý snímek ať se jedná o buňku, vojáky nebo útočící linky. Celá hra běží plynule přes funkci `requestAnimationFrame`, která zajistí nejrychlejší překreslení pro prohlížeč.

Fungování buněk: Každá buňka se nachází v kódu, jako objekt, který obsahuje důležité informace např. uchovává pozici na plátně, její barvu, aktuální počet vojáků, maximální počet vojáků. Vlastník buňky může mít tři scénáře, a to jest hráče, počítač (ai) a neutrální modrou buňku. Buňka se

zobrazuje jako kruh dané barvy s číslem vojáků, které ukazuje momentální počet vojáků. Číslo se nachází uprostřed buňky.

Fungování vojáků: Voják se pohybuje pouze od jedné buňky k druhé. Po vytvoření linky hráčem je odeslán voják z jedné buňky ke druhé. Vojáci jsou odesílání automaticky každých 500 milisekund. Voják se pohybuje s rychlostí 4 pixely za snímek. Voják má tři barvy bílá je pro bílou buňku, oranžová pro černou. Při dorážení buňky k počítači nastanou dvě situace, pokud má buňka stejnou barvu voják buňku posílí, ovšem pokud voják má jinou barvu než buňka, ztratí buňka život za každého vojáka. Při dobytí změni buňka barvu a přejde pod kontrolu nepřítele se začátečními 5 životy.

Fungování linek: Linka je trvalé spojení mezi dvěma buňkami, na které jsou odesílání vojáci. Hráč vytvoří linku tak že klikne na vlastní bílou buňku a poté klikne na cílovou buňku. Na plátně se linka zobrazí jako bílá čára mezi buňkami. Kliknutím přímo na linku a buňku ji hráč zruší. Pokud je dobytá buňka ze které linka vychází nebo buňka na kterou linka míří jsou všechny její linky automaticky zrušeny. Hráč může mít aktivních více linek zároveň což mu umožňuje útočit na více buněk současně nebo posilovat obranu vlastních buněk.

Fungování regenerace buněk: Buňka samostatně zregeneruje každých 800 milisekund. Buňka nesmí v tomto čase vysílat, ani dostat žádného vojáka k sobě. Hráč může využívat tuto funkci, jako taktiku pro regenerování buněk. Tento mechanismus platí pro všechny typy buněk bez ohledu na barvu.

Nepřítel: Počítačový nepřítel, neboli černá buňka má své chování přímočaré. Černá buňka vyšle každých 800 milisekund vojáka na náhodnou cílovou buňku. Černá buňka musí mít pro poslání vojáka aspoň 5 životů. Vybírání buňky závisí jen na náhodě. Černá buňka může útočit na modrou i bílou buňku.

Fungování animace výhry: Následně po splnění úrovně se na hráče spustí zlaté konfety. Každý snímek vytvoří s 33 procentní pravděpodobností vytvoří zlatý konfet na náhodném místě na horním okraji plátna. Každý konfet má různou rychlost a velikost. Animace nezavře výherní obrazovku a nevrátí se do hlavního menu. Tato animace slouží jako vizuální odměna pro hráče za úspěšné dokončení úrovně.

3.4 Backend - popis serverové části

Backend je aplikace, která běží v pozadí na serveru. Hráč backend nevidí, ale komunikuje s ním např. při přihlášení nebo při hraní v multiplayeru. Backend je postaven na frameworku Django, který je napsán v programovacím jazyce Python.

Fungování registrace a přihlášení: Při vyplnění registrace hráčem klikne na tlačítko a prohlížeč odešle jeho údaje na server. Server nejprve zkontroluje délku hesla s maximem 30 znaků. Poté kontroluje bezpečnost hesla a znovu jeho délku. Po kontrole se heslo zašifruje a spolu s ostatními údaji se uloží do databáze. Heslo se neukládá jako čistý text, ale šifruje se. Přihlášení funguje obdobně, hráč zadá uživatelské jméno nebo email a heslo, server najde hráče v databázi, zašifruje zadané heslo stejným způsobem a porovná ho s uloženým. Po přihlášení může hráč začít hrát.

Fungování databáze: Databáze je místo kde jsou trvale uloženy všechny informace o hráčích a jejich hrách. V projektu je použita SQLite databáze která ukládá vše do jednoho souboru na serveru. Databáze uchovává v různých tabulkách jiný typ dat. Tabulky např. uchovávají informace o registracích hráče. Tabulka herních relací informace o každé odehrané hře ve formátu JSON. Tabulka herních akcí jednotlivé akce hráče.

Fungování zabezpečení: Zabezpečení na několika úrovních pro chránění dat hráčů. Chránění probíhá formulářem CSFR tokenem, což je speciální kód který zabraňuje útokům kdy by cizí stránka mohla odesílat formuláře jménem přihlášeného hráče bez jeho vědomí. Evidence hesel je pomocí bezpečnostních pravidel Djanga a uložena v zašifrované podobě. Všechny citlivé operace jako je změna hesla nebo změna uživatelských údajů jsou navíc chráněny dodatečnou kontrolou oprávnění.

Fungování serverových funkcí: Všechny serverové funkce jsou napsány tak aby server mohl zpracovávat více požadavků současně aniž by musel čekat na dokončení předchozího. Tento proces pomáhá, aby mohli komunikovat více hráčů se serverem v jeden moment. Největší výhodou přináší tato funkce v multiplayerovém režimu. každá funkce přijme požadavek od prohlížeče, zpracuje ho, provede potřebné operace s databází a odešle odpověď zpět. Odpověď je vždy jednoduchý text například slovo success pokud operace proběhla úspěšně nebo slovo fail pokud nastala chyba.

3.5 Komunikace frontendu a backendu

Frontend a backend dvě oddělené části, které musejí spolu komunikovat. Bez této komplikace by nefungovaly základní funkce jako přihlášení, nebo multiplayer

Klasická HTTP komunikace: HTTP je základní způsob komunikace na internetu, který používá každý webový prohlížeč. Funguje, jako dopis a prohlížeč pošle požadavek na server a server odpoví. Tento způsob komunikace se používá pro všechny standardní operace, jako je registrace, přihlášení, odhlášení nebo načtení stránky. Nevýhoda HTTP je, že vždy musí prohlížeč první poslat požadavek. Server nemůže sám od sebe poslat zprávu prohlížeči. Pro obyčejné operace to nevadí, ale pro real-time hru, kde potřebujeme okamžitou komunikaci v obou směrech, to nestačí. Každý HTTP požadavek má svoji vlastní URL adresu, která je definována v souboru ``urls.py`` a přiřazena konkrétní funkci v souboru ``views.py``.

WebSocket komunikace: WebSocket je modernější způsob komunikace, který řeší nevýhodu HTTP. Po navázání spojení mohou obě strany, jak prohlížeč, tak server, posílat zprávy kdykoliv bez čekání na druhou stranu. Funguje to jako telefonní hovor a obě strany mohou mluvit kdykoliv chtějí, bez toho, aby museli čekat na otázku druhé strany. Toto je klíčové pro multiplayerový režim, kde hráč musí okamžitě vidět co udělal soupeř. WebSocket spojení zůstává otevřené po celou dobu hry a uzavře se až když hráč opustí herní stránku, nebo zavře prohlížeč.

Komunikace v praxi: Celý průběh komunikace během hraní multiplayeru funguje takto. Nejprve hráč vytvoří novou hru přes klasický HTTP požadavek a server mu vrátí číslo herní relace. Poté se prohlížeč hráče připojí na speciální WebSocket adresu s tímto číslem. Druhý hráč udělá totéž a připojí se na stejnou adresu. Od tohoto momentu jsou všechny akce ve hře pro vytváření linek, dobývání buněk a odesílány přes WebSocket na server. Server tyto zprávy okamžitě přepošle druhému hráči. Díky tomu oba hráči vidí stejnou herní situaci ve stejný čas, bez jakéhokoliv znatelného zpoždění. Django Channels zajišťuje správu všech WebSocket spojení, server Daphne umožňuje jejich souběžné zpracování a Redis slouží jako rychlý prostředník pro přeposílání zpráv mezi hráči.

4. Grafika a design

4.1 Grafický koncept buněk

Základní herní logika rozděluje buňky do tří typů na základě barev: bílé (ovládané hráčem), černé (počítač) a modré (neutrální). Aby hra působila profesionálnějším a organičtějším dojmem, byly tyto základní barevné kruhy nahrazeny detailními 2D texturami. Každý typ buňky dostal svůj unikátní vizuální charakter – od klidných a hladkých tvarů pro hráče a neutrální entity až po agresivnější, špičaté okraje pro buňky nepřátelské.

4.2 Technické zpracování textur

Při tvorbě grafických podkladů bylo striktně dodržováno pravidlo mocnin dvou (tzv. Power of Two). Herní enginey a webové prohlížeče dokážou textury v těchto rozměrech zpracovávat mnohem efektivněji. Rozlišení a formát: Všechny hlavní buňky byly nakresleny ve čtvercovém rozlišení 256x256 pixelů. Následně jsou přímo v kódu hry zmenšovány na požadovaný herní poloměr.

Tento přístup (kreslení ve vyšším rozlišení a následné zmenšení) zaručuje, že grafika zůstane na moderních monitorech naprosto ostrá. Pro zachování transparentního pozadí a poloprůhledných zářivých okrajů byl použit formát PNG (s alfa kanálem).

Ochranná zóna (Padding): Při kreslení textur byl záměrně ponechán volný prostor mezi samotnou buňkou a okrajem plátna. Tento prostor slouží jako rezerva pro animace (např. pulzování), aby nedocházelo k nežádoucímu ořezávání okrajů grafiky při jejím zvětšení během animační smyčky.

4.3 Animace pomocí Spritesheetů

Aby buňky nepůsobily staticky, byly do hry implementovány animace. HTML5 Canvas nativně nepodporuje přehrávání animovaných formátů, jako je GIF, aniž by došlo ke ztrátě kvality a průhlednosti. Z tohoto důvodu byla zvolena průmyslová metoda tzv. Spritesheetů.

Všechny fáze pohybu (snímky) dané animace byly exportovány a poskládány vedle sebe do jednoho dlouhého PNG obrázku. Program následně v herní smyčce (pomocí funkce `requestAnimationFrame`) posouvá výřez, kterým na tento pás nahlíží, čímž vzniká iluze plynulého pohybu. Tento způsob je extrémně paměťově nenáročný a zachovává dokonalou ostrost poloprůhledných stínů.

4.4 Grafika vojáků (Soldiers)

Vojáci představují jednotky, které se plynule pohybují od jedné buňky k druhé. V pokročilých fázích hry jich mohou být na plátně desítky až stovky současně. Z důvodu optimalizace výkonu bylo pro vojáky zvoleno menší rozlišení plátna, a to 32x32 pixelů.

Grafika vojáků je vycentrována přesně na střed, aby nedocházelo k vizuálním odchylkám při jejich pohybu po vypočítaných matematických linkách. Vizuál střel je zjednodušený, avšak zachovává jasnou barevnou identifikaci příslušnosti (hráč, nepřítel).

4.5 Prostředí a bezešvé pozadí

Pro umocnění mikroskopického dojmu (tzv. "Petriho miska" efekt) bylo navrženo organické pozadí. Místo použití jednoho obrovského statického obrázku, který by nadměrně zatěžoval operační paměť, byla vytvořena opakující se (seamless/ bezešvá) textura o velikosti 512x512 pixelů.

Při její tvorbě byla využita metoda posunu (offset), která zajistila, že na sebe okraje obrázku plynule navazují ve všech směrech. Pomocí metody `createPattern` na herním plátně se tato dlaždice donekonečna opakuje a vyplní celou hrací plochu, aniž by hráč zaznamenal jakékoliv přechody.

5. Využití AI při vývoji

Při vývoji našeho projektu jsme využili nástroje, které nám umožňují umělá inteligence. Umělou inteligenci jsme používali čistě, jako pomocný prostředek pro řešení problému, se kterým jsme si nevěděli rady. Hlavním nástrojem byl jazykový model Claude od společnosti Anthropic a Chat gpt od společnosti Open AI. Tyto nové funkce jsme využili, jako výdobytek moderní technologie.

- 1) Prvním naším problémem byla matematická funkce pro detekci bodu u úsečky. Tato funkce používala přesný výpočet vzdálenosti bodu od úsečky s pomocí vektorové projekce, skalárního součinu a parametrizace přímky. Tato část kódu se jedná o čistě matematické řešení.

Funkce používá vektorovou projekci pro výpočet vzdálenosti bodu od úsečky. Nejdřív se vypočítají dva vektory vektor v reprezentuje v kódu směr úsečky od a do b a vektor w reprezentuje směr od a k testovanému bodu. Pomocí skalárního součinu $c1 = v \cdot w$ a $c2 = v \cdot v$ se získá parametr $t = c1/c2$, který ukazuje, kde se nachází na úsečce nejbližší bod. $T=0$ znamená bod a , $t=1$ znamená bod b . Pomocí metod $\text{Math.max}(0, \text{Math.min}(1, t))$ se ořízne parametr na interval $<0,1>$, aby nejbližší bod nevypadl mimo úsečku. Poslední je Math.hypot změří euklidovskou vzdálenost mezi testovaným bodem a projekcí a porovná ji s `threshold`.

```
function isPointNearLine(x, y, a, b, threshold = 6) {  
  const vx = b.x - a.x;  
  const vy = b.y - a.y;  
  const wx = x - a.x;  
  const wy = y - a.y;  
  const c1 = vx * wx + vy * wy;  
  const c2 = vx * vx + vy * vy;  
  let t = 0;  
  if (c2 !== 0) t = c1 / c2;  
  t = Math.max(0, Math.min(1, t));  
  const px = a.x + t * vx;  
  const py = a.y + t * vy;  
  const dist = Math.hypot(x - px, y - py);  
  return dist <= threshold;  
}
```

obrázek 3

- 2) Druhým příkladem je funkce pro bezpečné načítání obrázků pomocí Promise. Funkce ošetřuje tři různé stavy, jako okamžitě načtený obrázek, úspěšné načtení i chybu. Funkce obalí nativní DOM events do Promise vzoru tzv. Promisification. Objekt Image má vlastnost `complete`, která je `true`. V případě zpracování obrázku z cache, v tomto případě se `resolve()` zavolá stejně, jinak se zaregistrují dva event listenery `load` pro úspěšné načtení a `error` pro selhání.

Oba zavolají `resolve()`, protože chceme pouze dokončení operace. Funkce se volá v `Promise.all()`, který čeká na všechny Promise souběžně a potom spustí `draw()`.

```
function nactiObrazek(img) {
  return new Promise(function(resolve) {
    if (img.complete) {
      resolve();
      return;
    }
    img.addEventListener('load', function() {
      resolve();
    });
    img.addEventListener('error', function() {
      console.warn("Obrázek se nepodařilo načíst: " + img.src);
      resolve();
    });
  });
}
```

obrázek 4

- 3) Třetím příkladem je systém pravidelného odesílání vojáků mezi buňkami pomocí `setInterval`. Kód v mezičase ukládá referenci na intervaly pro pozdější rušení a kontroluje duplicitu pomocí `some`. Funkce používá publisher pattern, kde každá linka je stálá `setInterval` s frekvencí 500 ms. Před vytvořením nové linky se pomocí `Array.some()` kontroluje existence duplicitní linky porovnáním referencí objektů `from` a `to`. Neporovnávají se jejich hodnoty. Reference na `intervalId` se ukládá do objektu v poli `activeLinks`, aby bylo možné interval, kdykoliv zrušit pomocí `clearInterval`. Bez uložení `intervalId` by interval běžel dál v paměti i po odebrání linky ze seznamu. Podmínka `if (!gamePaused)` uvnitř callbacku zajišťuje že interval sice tiká ale vojáci se odesílají pouze když hra běží.

```
function startAutoSend(from, to) {
  if (activeLinks.some(function(link) {
    return link.from === from && link.to === to;
  })) return;

  const interval = setInterval(function() {
    if (!gamePaused) {
      sendSoldiers(from, to);
    }
  }, 500);

  activeLinks.push({ from: from, to: to, interval: interval });
}
```

obrázek 5

6. Podobné hry srovnání

6.1 Podobné hry na trhu

Herní žánr pro hru Brain Cell Wars se nazývá RTS, neboli real-time strategy. Ve volném překladu znamená strategie v reálném čase. Existuje spousta her na trhu. Podobná je také hra Spore ve své počáteční buněčné fázi, kde hráč rovněž ovládá jednobuněčný organismus. V mobilním světě existují hry jako Galcon nebo Tentacle Wars, které mají velmi podobný princip vysílání vojáků mezi planetami nebo buňkami. Všechny tyto hry sdílejí základní myšlenku. Ovládání území prostřednictvím vysílání jednotek.

7. Závěr

7.1 Co jsme se naučili

Práce na projektu Brain Cell Wars nám přinesla cenné zkušenosti napříč celým spektrem webového vývoje. Na straně frontendu jsme se naučili pracovat s HTML5 Canvas a vykreslovat herní objekty pomocí JavaScriptu, implementovat herní smyčku pomocí requestAnimationFrame a navrhovat systém ukládání dat v prohlížeči. Pochopili jsme, jak funguje komunikace mezi více JavaScript soubory a i správně řídit pořadí jejich načítání. Na straně backendu jsme získali první praktické zkušenosti s frameworkem Django a programovacím jazykem Python. Naučili se navrhovat databázové modely, psát asynchronní serverové funkce a implementovat zabezpečení webové aplikace. Největší výzvou a zároveň největším přínosem bylo propojení frontendu s backendem přes WebSocket protokol pro multiplayerový režim. Tato technologie nám otevřela pohled na to, jak fungují moderní real-time webové aplikace.

7.2 Co by šlo vylepšit

Projekt má několik oblastí, které by bylo možné v budoucnu vylepšit. Z hlediska hratelnosti by bylo zajímavé přidat chytřejší umělou inteligenci počítačového nepřítele, která by útočila strategicky místo náhodně. Dalším vylepšením by mohlo být přidání více typů buněk s různými schopnostmi nebo speciálních útoků. Z technického hlediska by bylo vhodné přesunout ukládání herního postupu z `localStorage` na server, což by umožnilo hráčům hrát ze více zařízení a jejich postup by byl vždy synchronizován. Grafická stránka hry by mohla být dále rozvinuta přidáním plynulých animací buněk pomocí `spritesheet` technologie, kterou jsme v projektu začali implementovat ale plně nedokončili.

7.3 Zhodnocení projektu

Projekt Brain Cell Wars splnil všechny cíle, které jsme si na začátku stanovili. Podařilo se nám vytvořit funkční prohlížečovou hru s třemi herními úrovněmi, systémem přihlašování a ukládání postupu a připravenou infrastrukturou pro multiplayerový režim. Projekt nás posunul výrazně dopředu v programátorských dovednostech a ukázal nám, jak vypadá vývoj skutečné webové aplikace od návrhu až po implementaci. Naučili jsme se pracovat v týmu, rozdělovat práci a řešit problémy, které přicházely během vývoje.

Zdroje

- Django SOFTWARE FOUNDATION. Cross Site Request Forgery protection. In: Django documentation [online]. Dostupné z: <https://docs.djangoproject.com/en/6.0/ref/csrf/> [cit. 2025-12-12].
- Django SOFTWARE FOUNDATION. Built-in template tags and filters. In: Django documentation[online].Dostupné z:<https://docs.djangoproject.com/en/6.0/ref/templates/builtins/> [cit. 2025-12-20].
- Django SOFTWARE FOUNDATION. Sending email. In: Django documentation [online]. Dostupné z: <https://docs.djangoproject.com/en/6.0/topics/email/> [cit. 2026-01-05].
- Django SOFTWARE FOUNDATION. Asynchronous support. In: Django documentation [online]. Dostupné z: <https://docs.djangoproject.com/en/6.0/topics/async/> [cit. 2026-01-18].
- Django SOFTWARE FOUNDATION. URL dispatcher. In: Django documentation [online]. Dostupné z: <https://docs.djangoproject.com/en/6.0/topics/http/urls/> [cit. 2026-02-02].
- Django SOFTWARE FOUNDATION. Views. In: Django documentation [online]. Dostupné z: <https://docs.djangoproject.com/en/6.0/topics/http/views/> [cit. 2026-02-10].
- Django SOFTWARE FOUNDATION. View decorators. In: Django documentation [online]. Dostupné z: <https://docs.djangoproject.com/en/6.0/topics/http/decorators/> [cit. 2026-02-18].
- Django SOFTWARE FOUNDATION. Database queries. In: Django documentation [online]. Dostupné z: <https://docs.djangoproject.com/en/6.0/topics/db/queries/> [cit. 2026-02-25].
- Django SOFTWARE FOUNDATION. Aggregation. In: Django documentation [online]. Dostupné z: <https://docs.djangoproject.com/en/6.0/topics/db/aggregation/> [cit. 2026-03-03].
- Django SOFTWARE FOUNDATION. Transactions. In: Django documentation [online]. Dostupné z: <https://docs.djangoproject.com/en/6.0/topics/db/transactions/> [cit. 2026-03-10].
- Django SOFTWARE FOUNDATION. Optimization. In: Django documentation [online]. Dostupné z: <https://docs.djangoproject.com/en/6.0/topics/db/optimization/> [cit. 2026-03-18].
- Django SOFTWARE FOUNDATION. Fixtures. In: Django documentation [online]. Dostupné z: <https://docs.djangoproject.com/en/6.0/topics/db/fixtures/> [cit. 2026-03-25].
- Django SOFTWARE FOUNDATION. Request and response objects.In:Django documentation[online].Dostupnéz:<https://docs.djangoproject.com/en/6.0/ref/request-response/> [cit. 2026-04-01].
- Django SOFTWARE FOUNDATION. Model field reference. In: Django documentation [online]. Dostupné z: <https://docs.djangoproject.com/en/6.0/ref/models/fields/> [cit. 2026-04-08].
- Django SOFTWARE FOUNDATION. Settings. In: Django documentation [online]. Dostupné z: <https://docs.djangoproject.com/en/6.0/ref/settings/> [cit. 2026-04-15].
- Django SOFTWARE FOUNDATION. Authentication system. In: Django documentation [online]. Dostupné z: <https://docs.djangoproject.com/en/6.0/ref/contrib/auth/> [cit. 2026-04-20].
- Django SOFTWARE FOUNDATION. Using the authentication system. In: Django documentation [online].Dostupné z: <https://docs.djangoproject.com/en/6.0/topics/auth/default/> [cit. 2026-04-25].

- Django SOFTWARE FOUNDATION. Customizing authentication. In: Django documentation [online]. Dostupné z: <https://docs.djangoproject.com/en/6.0/topics/auth/customizing/> [cit. 2026-04-30].
- Django Channels Django Channels CHANNELS PROJECT. Tutorial. In: Channels documentation[online].Dostupné z: <https://channels.readthedocs.io/en/latest/tutorial/index.html> [cit. 2026-05-01].
- Django Channels CHANNELS PROJECT. Database access. In: Channels documentation [online]. Dostupné z: <https://channels.readthedocs.io/en/latest/topics/databases.html>[cit. 2026-05-03].
- Mozilla MOZILLA. *Canvas API*. In: MDN Web Docs [online]. Dostupné z: https://developer.mozilla.org/en-US/docs/Web/API/Canvas_API [cit. 2025-12-15].
- Mozilla MOZILLA. *Window: requestAnimationFrame() method*. In: MDN Web Docs [online]. Dostupné z: <https://developer.mozilla.org/en-US/docs/Web/API/Window/requestAnimationFrame> [cit. 2025-12-22].
- Mozilla MOZILLA. *Window: localStorage property*. In: MDN Web Docs [online]. Dostupné z: <https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage> [cit. 2026-01-08].
- Mozilla MOZILLA. *WebSocket API*. In: MDN Web Docs [online]. Dostupné z: https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API [cit. 2026-01-20].
- Django Channels CHANNELS PROJECT. *Introduction*. In: Channels documentation [online]. Dostupné z: <https://channels.readthedocs.io/en/stable/introduction.html> [cit. 2026-01-28].
- Mozilla MOZILLA. *Promise.all()*. In: MDN Web Docs [online]. Dostupné z: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise/all [cit. 2026-02-05].
- Mozilla MOZILLA. *setInterval() global function*. In: MDN Web Docs [online]. Dostupné z: <https://developer.mozilla.org/en-US/docs/Web/API/setInterval> [cit. 2026-02-12].
- Django DJANGO SOFTWARE FOUNDATION. *Django documentation*. [online]. Dostupné z: <https://docs.djangoproject.com/en/stable/> [cit. 2026-02-20].
- Mozilla MOZILLA. *CanvasRenderingContext2D: globalCompositeOperation property*. In: MDN Web Docs [online]. Dostupné z: <https://developer.mozilla.org/en-US/docs/Web/API/CanvasRenderingContext2D/globalCompositeOperation> [cit. 2026-03-01].
- **ChatGPT — konverzační AI** OpenAI. *ChatGPT*. Dostupné z: <https://chat.openai.com/> [cit. 2026-03-18]
- **Claude — konverzační AI** Anthropic. *Claude AI*. Dostupné z: <https://claude.ai/> [cit. 2025-12-09]

Obrázky

Obrázek 1 - vlastní zdroj

Obrázek 2 - vlastní zdroj

Obrázek 3 - vlastní zdroj

Obrázek 4 - vlastní zdroj